

Sprint 3 — Catálogo de Serviços Protegido com JWT

1. Objetivos do trabalho

- Criar o modelo Service;
- Implementar o CRUD de serviços;
- Permitir criar, listar, consultar, editar e apagar tipos de reparações da oficina;
- Proteger todos os endpoints de serviços com middleware JWT;
- Garantir que apenas utilizadores autenticados conseguem gerir o catálogo de serviços;
- Atualizar ou implementar a documentação OpenAPI/Swagger com os endpoints de serviços;
- Testar as rotas protegidas no Insomnia;

2. Configuração da aplicação

O ficheiro principal da aplicação é o index.js. Este ficheiro tem como função:

- carregar as variáveis de ambiente;
- ligar à base de dados MongoDB;
- configurar o Express;
- ativar o cors;
- permitir o uso de JSON nos pedidos;
- carregar a documentação OpenAPI;
- registar as rotas de autenticação;
- registar as rotas de serviços;
- iniciar o servidor na porta definida.

A aplicação fica disponível em:

<http://127.0.0.1:3000>

A documentação Swagger fica disponível em:

<http://localhost:3000/api-docs>

3. Modelo Service

Foi criado o modelo Service, responsável por representar os serviços disponíveis na oficina.

O modelo contém os seguintes campos:

Campo	Tipo	Descrição
name	String	Nome do serviço, por exemplo “Mudança de Óleo”
description	String	Descrição do serviço
estimated_time	Number	Tempo estimado do serviço, em minutos
createdAt	Date	Data automática de criação
updatedAt	Date	Data automática de atualização

O campo name foi definido como único, de forma a evitar a criação de serviços duplicados com o mesmo nome.

4. Implementação do CRUD de Serviços

O CRUD de serviços foi implementado no ficheiro:

src/controllers/serviceController.js

Foram implementadas as seguintes operações:

Operação	Função	Descrição
Listar serviços	getServices	Devolve todos os serviços existentes
Consultar serviço por ID	getServiceById	Devolve um serviço específico
Criar serviço	createService	Cria um novo serviço
Atualizar serviço	updateService	Atualiza os dados de um serviço
Apagar serviço	deleteService	Remove um serviço da base de dados

5. Endpoints de serviços implementados

Foram implementados os seguintes endpoints:

Método	Endpoint	Descrição	Proteção
GET	/api/services	Listar todos os serviços	JWT
POST	/api/services	Criar novo serviço	JWT
GET	/api/services/:id	Consultar serviço por ID	JWT

Método	Endpoint	Descrição	Proteção
PUT	/api/services/:id	Atualizar serviço por ID	JWT
DELETE	/api/services/:id	Apagar serviço por ID	JWT

Todos estes endpoints estão protegidos pelo middleware JWT.

6. Middleware de autenticação JWT

A proteção dos endpoints foi feita com o middleware:

src/middlewares/authMiddleware.js

Este middleware verifica se o pedido contém o cabeçalho:

Authorization: Bearer TOKEN_JWT

Se o token não for enviado, a API devolve uma resposta de erro

7. Documentação OpenAPI/Swagger

A documentação OpenAPI foi implementada através do ficheiro:

src/docs/openapi.yaml

Para permitir a visualização da documentação, foram usadas as dependências:

swagger-ui-express

yamljs

Estas dependências permitem carregar o ficheiro openapi.yaml e disponibilizar a documentação no browser através da rota:

http://localhost:3000/api-docs

8. Testes no Insomnia

O ficheiro do Insomnia apresenta uma coleção de testes do Insomnia, contém os pedidos necessários para testar:

- Registo de utilizador;
- Login;
- Obtenção do token JWT;
- Criação de serviço;
- Listagem de serviços;
- Consulta de serviço por ID.